

SingleCell workflow methods

Stepwise single-cell RNA-seq analysis for wet-lab users

SingleCell pipeline

2026-06-22

- Overview
- Computational environment
- Chapter 1 - Single-cell processing
 - Sample definition
 - Loading and pre-filter QC
 - Filtering and doublet detection
 - Integration, Harmony and clustering
 - Marker-based and reference-based annotation
 - Optional curation and export
- Chapter 2 - Pseudobulk, DESeq2, GO and hdWGCNA
 - Cell-type subsets
 - Pseudo-replicate assignment
 - Pseudobulk aggregation and DESeq2
 - Volcano plots
 - Differential gene tables
 - GO enrichment
 - Log2FC heatmap
 - Unified hdWGCNA on significant genes
 - Network export and visualization
- Chapter 3 - Pseudotime
 - Load curated AnnData object
 - Select cell types
 - Infer trajectory
 - Plot genes on the trajectory
 - Gene trends
- Principal helper functions
 - R helper functions
 - Python helper functions
- Output structure
- Reproducibility notes

Overview

This document describes the current SingleCell workflow as a methods-style protocol. The pipeline is organized into three chapters:

1. **Chapter 1 - Single-cell processing:** load raw matrices, quality control, filtering, doublet removal, integration, clustering, cell-type annotation and export of the curated object.
2. **Chapter 2 - Pseudobulk, DESeq2 and networks:** create cell-type subsets, assign pseudo-replicates, run DESeq2, render volcano plots, build differential expression tables, run GO enrichment, build log2FC heatmaps and infer a unified hdWGCNA network from significant genes.
3. **Chapter 3 - Pseudotime:** load the curated AnnData object, select cell types, infer a trajectory with Palantir and scFates, plot genes on the trajectory and generate gene trend heatmaps.

The workflow is intentionally stepwise. A basic user should mainly edit the small parameter blocks inside each section, not the helper functions.

Computational environment

The analysis is designed to run inside the Docker image:

```
docker run -it -v ~/projects:/workspace matigara/scrnaseq:latest /bin/bash
```

The workflow scripts assume that the pipeline lives in:

```
/workspace/workflow_nuevo
```

and that output is written to:

```
/workspace/resultados
```

The R chapters load:

```
PIPELINE_DIR <- "/workspace/workflow_nuevo"
DATA_DIR     <- "/workspace/."
base_dir     <- file.path(DATA_DIR, "resultados")

source(file.path(PIPELINE_DIR, "load_libraries.R"))
source(file.path(PIPELINE_DIR, "custom_seurat.R"))
source(file.path(PIPELINE_DIR, "ScRNA_Analysis_Functions.R"))
```

The Python chapter loads:

```
PIPELINE_DIR = "/workspace/workflow_nuevo"
DATA_DIR      = "/workspace/."
base_dir      = os.path.join(DATA_DIR, "resultados")

exec(open(os.path.join(PIPELINE_DIR, "load_libraries_python.py")).read(), globals())
exec(open(os.path.join(PIPELINE_DIR, "ScRNA_Pseudotime_Functions.py")).read(),
      globals())
```

Chapter 1 - Single-cell processing

Chapter 1 produces the curated Seurat object used by the rest of the pipeline. It is implemented in:

```
workflow/capitulo1_single_cell.R
```

Sample definition

The user edits the sample list and colors at the beginning of the chapter. Each sample points to a raw matrix file and defines the biological condition.

```
samples <- list(
  list(file = "sample_1/filtered_feature_bc_matrix.h5",
        label = "Sample_1",
        condition = "condition_1"),
  list(file = "sample_2/filtered_feature_bc_matrix.h5",
        label = "Sample_2",
        condition = "condition_2")
)

colors <- c("condition_1" = "#3B82F6",
            "condition_2" = "#EF4444")
```

Loading and pre-filter QC

The function `load_seurat_samples()` loads all samples and computes quality metrics such as mitochondrial and chloroplast percentages.

```
mt_pattern <- "^ATMG"
cp_pattern <- "^ATCG"

seurat_list_raw <- load_seurat_samples(
  samples      = samples,
  DATA_DIR    = DATA_DIR,
  USE_CELLBENDER = FALSE,
  mt_pattern    = mt_pattern,
```

```
cp_pattern = cp_pattern
)
```

Filtering and doublet detection

The function `filter_seurat_samples()` applies simple QC thresholds and runs the sample-level filtering logic.

```
seurat_list <- filter_seurat_samples(
  seurat_list_raw,
  min_features = 200,
  max_mt       = 5
)
```

Integration, Harmony and clustering

The merged object is normalized, scaled, reduced by PCA and corrected with Harmony. Clustering is then selected from a user-defined resolution.

```
pbmc_harmony <- reduce(seurat_list, merge)
pbmc_harmony <- NormalizeData(pbmc_harmony)
pbmc_harmony <- FindVariableFeatures(pbmc_harmony)
pbmc_harmony <- ScaleData(pbmc_harmony)
pbmc_harmony <- RunPCA(pbmc_harmony)
pbmc_harmony <- RunHarmony(pbmc_harmony, group.by.vars = "orig.ident")
```

```
cluster_resolution <- 0.3

pbmc_harmony <- pbmc_harmony |>
  FindNeighbors(reduction = "harmony", dims = 1:30) |>
  FindClusters(resolution = cluster_resolution) |>
  RunUMAP(reduction = "harmony", dims = 1:30)
```

Marker-based and reference-based annotation

The function `find_markers()` identifies cluster markers. The function `annotate_by_markers()` applies a manual marker table. When a reference object is available, `annotate_by_reference()` transfers cell-type labels.

```
markers <- find_markers(
  pbmc_harmony,
  output_dir = dir_03
)

pbmc_harmony <- annotate_by_markers(
  pbmc_harmony,
  markers      = markers,
```

```
output_dir = dir_03
)
```

```
reference_obj <- readRDS(file.path(DATA_DIR,
  "GSE273033_seuratObj_for_publication.rds"))

pbmc_harmony <- annotate_by_reference(
  query_obj      = pbmc_harmony,
  reference_obj  = reference_obj,
  output_dir     = dir_03
)
```

Optional curation and export

The user can manually group or reassign cell types. The final curated object is saved as both RDS and H5AD.

```
saveRDS(pbmc_harmony, file.path(dir_objects, "pbmc_harmony_curated.rds"))

export_to_scanpy(
  seurat_obj = pbmc_harmony,
  output_h5ad = file.path(dir_objects, "pbmc_harmony_curated.h5ad")
)
```

Chapter 2 - Pseudobulk, DESeq2, GO and hdWGCNA

Chapter 2 starts from the curated Seurat object:

```
workflow/capitulo2_pseudobulk_de.R
```

Cell-type subsets

The analysis first creates one Seurat object per cell type using `create_cell_type_subsets()`.

```
pseudobulk_annot_col <- "celltype_curated"

cell_type_subsets <- create_cell_type_subsets(
  pbmc_harmony,
  annot_col = pseudobulk_annot_col
)
```

Pseudo-replicate assignment

Pseudo-replicates are assigned within each condition. The condition column is used for DESeq2 comparisons; `orig.ident` stores the original sample or library; `replicate` is created by this step.

```
pseudobulk_conditions <- NULL
n_pseudoreps          <- 3

cell_type_subsets_replicates <- assign_pseudoreplicates_batch(
  cell_type_subsets,
  pseudobulk_conditions = pseudobulk_conditions,
  n_pseudoreps          = n_pseudoreps
)
```

The user can check the three relevant metadata columns:

```
table(cell_type_subsets_replicates$Pavement_Cell$condition)
table(cell_type_subsets_replicates$Pavement_Cell$replicate)
table(cell_type_subsets_replicates$Pavement_Cell$orig.ident)
```

Pseudobulk aggregation and DESeq2

The function `run_pseudobulk_deseq2_analysis()` aggregates counts by pseudo-replicate and runs DESeq2 for each requested comparison.

```
comparisons <- list(
  list(conds = c("condicion1", "condicion2"),
        tag   = "condicion1_vs_condicion2")
)

cell_types_to_analyze <- NULL

deseq2_results <- run_pseudobulk_deseq2_analysis(
  cell_type_subsets_replicates = cell_type_subsets_replicates,
  comparisons                  = comparisons,
  output_dir                   = dir_06,
  cell_types                   = cell_types_to_analyze,
  pseudobulk_dir               = file.path(dir_objects, "pseudobulk_replicas")
)
```

Interpretation:

```
conds = c("reference", "treatment")
log2FC > 0 means higher expression in treatment
```

Volcano plots

The function `render_volcano_plots()` uses the DESeq2 tables from a selected contrast.

```
volcano_tag <- "condicion1_vs_condicion2"
padj_cut    <- 0.05
lfc_cut     <- 1

render_volcano_plots(
  results_dir = file.path(dir_06, volcano_tag),
  output_dir  = file.path(dir_06, volcano_tag, "volcano"),
  pdf_name    = paste0("VolcanoPlots_", volcano_tag, ".pdf"),
  padj_cut    = padj_cut,
  lfc_cut     = lfc_cut
)
```

Differential gene tables

The function `build_differential_tables()` creates the summary tables used by the heatmap and by the network step.

```
diff_tables <- build_differential_tables(
  results_dir = file.path(dir_06, volcano_tag),
  output_dir  = file.path(dir_06, volcano_tag),
  prefix      = paste0("diff_table_", volcano_tag),
  padj_cut    = padj_cut,
  lfc_cut     = lfc_cut
)
```

GO enrichment

GO enrichment is wrapped in `run_go_enrichment_for_contrast()`, so the chapter stays readable.

```
go_results <- run_go_enrichment_for_contrast(
  results_dir = file.path(dir_06, volcano_tag),
  output_dir  = file.path(dir_07, volcano_tag),
  orgdb       = org.At.tair.db,
  keytype     = "TAIR",
  go_space    = "BP",
  padj_cutoff = 0.05
)
```

Log2FC heatmap

The heatmap compares the differential expression pattern across cell types.

```
heatmap_limits <- c(-5, 5)

build_logfc_heatmap(
  logfc_table = file.path(dir_06, volcano_tag, "tabla_log2FC_fc1_padj_005.tsv"),
  output_file = file.path(dir_06, volcano_tag, "log2FC_heatmap.pdf"),
  color_limits = heatmap_limits
)
```

Unified hdWGCNA on significant genes

The function `run_unified_hdwgcna()` filters the Seurat object to the significant genes in the DE summary table and runs one unified hdWGCNA network. This is not one network per cell type; it is one network over the significant gene subset.

```
wgcna_name <- "unified"
n_metacells <- 25
soft_power <- NULL

run_unified_hdwgcna(
  seurat_obj = pbmc_harmony,
  de_table_path = file.path(dir_06, volcano_tag, "tabla_log2FC_fc1_padj_005.tsv"),
  output_dir = dir_08,
  annot_col = pseudobulk_annot_col,
  sample_col = "orig.ident",
  wgcna_name = wgcna_name,
  n_metacells = n_metacells,
  soft_power = soft_power
)
```

Network export and visualization

The function `plot_hdwgcna_network()` exports edge and node tables and draws the network. Genes without connections after the TOM threshold are not plotted.

```
tom_threshold <- 0.5
n_hub_label <- 5

plot_hdwgcna_network(
  hdwgcna_dir = dir_08,
  output_dir = dir_08,
  tom_threshold = tom_threshold,
  n_hub_label = n_hub_label
)
```

Interpretation:

Lower tom_threshold = more edges and more genes.
Higher tom_threshold = stronger edges and fewer genes.
Only connected genes are shown in the final plot.

Chapter 3 - Pseudotime

Chapter 3 is implemented in Python:

```
workflow/capitulo3_pseudotime.py
```

It uses:

```
workflow/ScRNA_Pseudotime_Functions.py
```

Load curated AnnData object

The function `load_curated_object()` reads the H5AD object from Chapter 1, standardizes coordinate names and saves an overview UMAP.

```
INPUT_H5AD      = os.path.join("/workspace/resultados/objects/  
                                pbmc_harmony_curated.h5ad")  
ANNOTATION_COL  = "celltype_curated"  
N_JOBS          = 4  
  
adata, N_JOBS = load_curated_object(  
    input_h5ad      = INPUT_H5AD,  
    dir_pseudotime  = dir_pseudotime,  
    annotation_col  = ANNOTATION_COL,  
    n_jobs          = N_JOBS,  
)
```

Select cell types

The function `preview_trajectory_selection()` subsets the selected cell types and saves a UMAP preview.

```
TRAJECTORY_CLUSTERS = ["Pavement Cell", "Stomatal Line"]  
  
adata_sub = preview_trajectory_selection(  
    adata          = adata,  
    clusters       = TRAJECTORY_CLUSTERS,  
    annotation_col = ANNOTATION_COL,  
    dir_pseudotime = dir_pseudotime,  
)
```

Infer trajectory

The function `trajectory_run()` defines one parameter set. The function `run_trajectory_runs()` builds the trajectory using Palantir diffusion maps and scFates PPT, selects a root cell and assigns pseudotime.

```
ROOT_CLUSTER = "Stomatal Line"

TRAJECTORY_RUNS = [
    trajectory_run(nodes=25, sigma=0.5, lambda_value=30, eigs=3, seed=3),
]

adata_traj, selected_trajectory_dir, trajectory_runs = run_trajectory_runs(
    adata          = adata,
    clusters       = TRAJECTORY_CLUSTERS,
    root_cluster   = ROOT_CLUSTER,
    annotation_col = ANNOTATION_COL,
    output_base_dir = dir_pseudotime,
    runs           = TRAJECTORY_RUNS,
)
```

Plot genes on the trajectory

Section 28 is intentionally plain Scanpy code. The user edits only the gene list.

```
GENES = ["AT1G22690", "AT4G04890", "AT1G80070"]

gene_plots_dir = os.path.join(selected_trajectory_dir, "gene_plots")
os.makedirs(gene_plots_dir, exist_ok=True)

fig = sc.pl.draw_graph(
    adata_traj,
    color      = GENES,
    title      = GENES,
    show       = False,
    return_fig = True,
)
fig.set_size_inches(18, 18, forward=True)
fig.savefig(os.path.join(gene_plots_dir, "step28_genes_FINAL_18x18.png"),
            dpi=200, facecolor="white")
fig.savefig(os.path.join(gene_plots_dir, "step28_genes_FINAL_18x18.pdf"),
            dpi=200, facecolor="white")
plt.close(fig)
```

Gene trends

The function `run_step29_gene_trends()` creates two heatmaps:

1. The top TOP_N trajectory-associated genes.
2. A customizable highlight list chosen by the user.

```

TOP_N                = 10
HIGHLIGHT_GENES     = ["AT1G22690", "AT4G04890", "AT1G80070"]
ORDERING             = "max"

top_path, highlight_path = run_step29_gene_trends(
    adata              = adata_traj,
    run_dir            = selected_trajectory_dir,
    name               = os.path.basename(selected_trajectory_dir),
    custom_genes       = HIGHLIGHT_GENES,
    top_n              = TOP_N,
    ordering           = ORDERING,
    n_jobs             = N_JOBS,
)

```

Principal helper functions

The workflow keeps complex implementation inside helper functions. The chapter scripts remain readable and mostly contain parameters.

R helper functions

```

load_seurat_samples()
filter_seurat_samples()
find_markers()
annotate_by_markers()
annotate_by_reference()
export_to_scanpy()
create_cell_type_subsets()
assign_pseudoreplicates_batch()
run_pseudobulk_deseq2_analysis()
render_volcano_plots()
build_differential_tables()
run_go_enrichment_for_contrast()
build_logfc_heatmap()
run_unified_hdwcna()
plot_hdwcna_network()

```

Python helper functions

```

load_curated_object()
preview_trajectory_selection()
trajectory_run()
build_pseudotime_trajectory()
run_trajectory_runs()
run_step29_gene_trends()

```

Output structure

The main output folders are:

```
resultados/  
├─ 01_qc/  
├─ 02_clustering/  
├─ 03_annotation/  
├─ 04_gene_expression/  
├─ 05_celltype_curation/  
├─ 06_pseudobulk_deseq2/  
├─ 07_go_enrichment/  
├─ 08_networks/  
├─ 09_pseudotime/  
└─ objects/
```

Reproducibility notes

All plot sizes are standardized to 18 x 18 where applicable. The most important user-edited parameters are kept directly inside each section. The document intentionally shows the relevant calls in their native language:

```
R      = Chapters 1 and 2  
Python = Chapter 3
```

This keeps the protocol close to how a wet-lab user actually executes the pipeline: step by step, changing only visible parameters.